

Practical Project

Image Processing - Vision Editor

Prepared By :

Mohab Nasser Abdelkader	23013228
Ali Mohamed Mahmoud	23015406
Youssef Amr Mohamed	23014943
Mostafa Waleed Hamed	23110006
Youssef Samir Abdelkader	23016427

Supervised By :

Dr. Hussam Elbehiery

Co-Supervisor :

Eng. Maryam Ali

Eng. Shady Bedier

Eng. Rahma Owaidah

Eng. Shimaa Anwar

Eng. Aya Ebrahim

1. Introduction

This report documents the design and implementation of an Image Processing Desktop Application developed as a team project. The application provides a graphical user interface (GUI) that allows users to load any JPEG or PNG image and apply eleven built-in image-processing operations — all without using the command line.

The project is built on three Python libraries:

- OpenCV (cv2) — provides all low-level image-processing algorithms.
- Tkinter — supplies the native desktop GUI (windows, buttons, sliders, canvases).
- Pillow (PIL) — bridges OpenCV NumPy arrays to Tkinter-compatible PhotoImage objects.
- Matplotlib — renders live histograms as images displayed inside the GUI canvases.

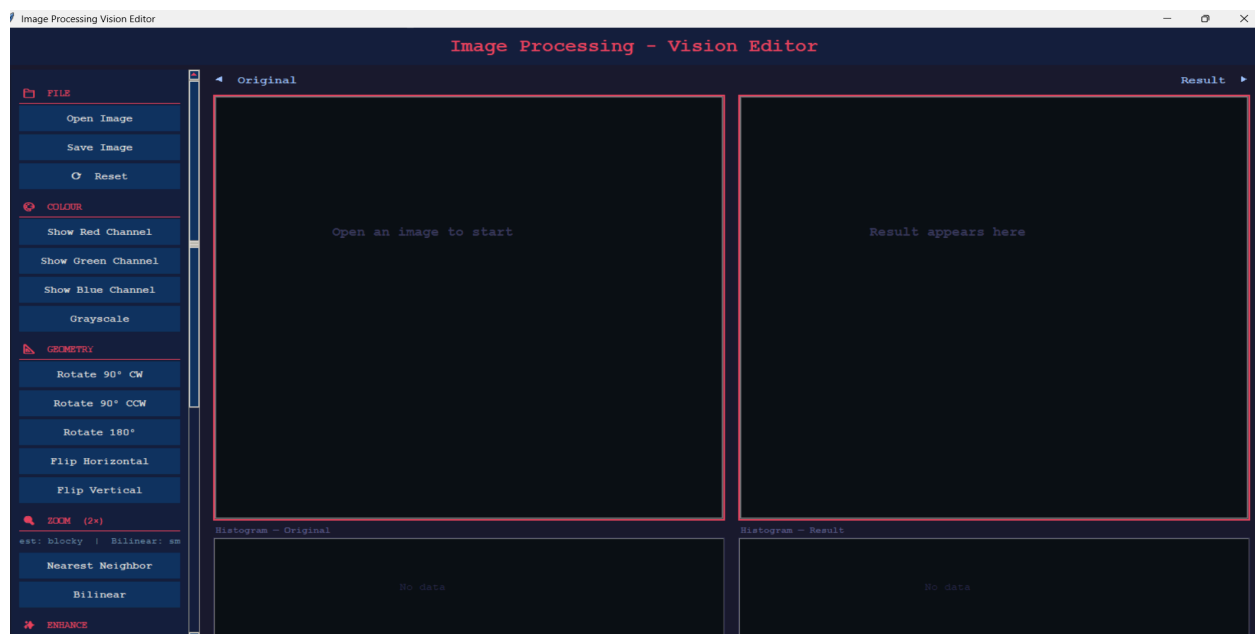
2. Application Architecture

The entire application is contained in a single Python file (newFinal.py) and follows an object-oriented design centred on the ImageProcessorApp class. When the program starts, the Tk root window is created, a ttk.Style is applied, and an instance of ImageProcessorApp is instantiated. All state and event handling live inside that class.

2.1 GUI Layout

The window is divided into three regions:

- Left Control Panel — a scrollable column of buttons and sliders, one per feature group.
- Right Image Area — two image canvases (Original and Result) with a live histogram canvas beneath each.
- Status Bar — a single line at the bottom that reports the last action taken.



2.2 Key Helper Functions

`cv2_to_tk(img, max_w, max_h)`

Converts any OpenCV image (BGR or grayscale) to a Tkinter PhotoImage object scaled to fit inside the canvas. It calls `cv2.cvtColor` to handle the BGR-to-RGB conversion that Pillow requires, then uses `PIL.Image.thumbnail` for aspect-ratio-safe resizing.

`make_histogram_image(img, width, height)`

Renders a Matplotlib figure off-screen using the non-interactive Agg backend. For grayscale images it plots a single grey curve; for colour images it plots three overlapping curves (red, green, blue) and fills them with translucent colour. The x-axis is fixed at 0–255 with a small padding on each side, and the y-axis is auto-scaled with 10 % headroom. The rendered figure is exported as a NumPy RGBA array, converted to BGR, resized to match the canvas, and returned for display.

`_display_result(img, update_current)`

Central method called by every processing feature. It stores the processed image as `self.current_img`, converts it to Tkinter format, draws it on the result canvas, and triggers `_update_histograms()` to refresh both mini-histogram strips.

3. Feature Explanations

The table below maps each required feature to its description and the primary OpenCV function used.

#	Feature	Description	OpenCV Function
1	Open Image	Loads a JPEG or PNG file from disk using a file dialog, reads it with OpenCV and displays it in the Original panel.	<code>cv2.imread()</code>
2	Show Channels	Splits the image into R, G, B components; isolates each channel by zeroing the other two and merges back.	<code>cv2.split()</code> , <code>cv2.merge()</code>
3	Grayscale	Converts a colour BGR image to a single-channel grayscale image using a weighted luminance formula.	<code>cv2.cvtColor(COLOR_BGR2GRAY)</code>
4	Geometric Transform	Zooms in 2x using either Nearest-Neighbor (blocky) or Bilinear (smooth) interpolation; also supports 90°/180° rotations.	<code>cv2.resize()</code> , <code>cv2.rotate()</code>
5	Flip	Mirrors the image horizontally (left-right) or vertically (up-down).	<code>cv2.flip()</code>

#	Feature	Description	OpenCV Function
6	Enhancement	Histogram Equalization redistributes pixel intensities for contrast. Gamma Correction applies a power-law transform (LUT) to brighten or darken.	<code>cv2.equalizeHist()</code> , <code>cv2.LUT()</code>
7	Gaussian Blur	Applies a Gaussian kernel of odd size (3–21) to smooth the image and reduce noise.	<code>cv2.GaussianBlur()</code>
8	Edge Detection	Converts to grayscale then runs the Canny algorithm with user-adjustable low/high thresholds to highlight edges.	<code>cv2.Canny()</code>
9	Compare/Histogram	Plots a live mini-histogram under each panel (intensity 0–255) and offers a full-size histogram popup.	<code>cv2.calcHist()</code> , <code>matplotlib</code>
10	Reset	Restores the currently displayed image to the original loaded image.	Array copy
11	Save Image	Saves the current result image to disk in PNG or JPEG format.	<code>cv2.imwrite()</code>

3.1 Open Image (Feature 1)

A `tk.filedialog.askopenfilename` dialog filters for common image extensions. The selected path is passed to `cv2.imread`, which returns a BGR NumPy array. The array is stored in both `self.original_img` (never modified) and `self.current_img` (the working copy). The status bar shows the image dimensions and channel count.

3.2 Show Channels (Feature 2)

`cv2.split` decomposes the BGR image into three single-channel arrays `b`, `g`, `r`. To display, for example, the Red channel, a zero array `z` is created and `cv2.merge([z, z, r])` builds a new BGR image where only the red channel carries data. The result appears tinted red in the display because the other channels are black.

3.3 Grayscale Conversion (Feature 3)

`cv2.cvtColor` with the flag `cv2.COLOR_BGR2GRAY` applies OpenCV's standard luminance-weighted formula ($0.299 R + 0.587 G + 0.114 B$) to produce a single-channel image. If the image is already grayscale the operation is skipped with a status message.

3.4 Geometric Transformation — Zoom & Rotate (Feature 4)

Zoom: the centre quarter of the image (half the width, half the height) is cropped, then upsampled back to the original dimensions using either `cv2.INTER_NEAREST` (replicates pixels —

produces a blocky, pixelated result) or `cv2.INTER_LINEAR` (bilinear interpolation — estimates new pixel values by weighting the four nearest neighbours, producing a smooth result).

Rotation: `cv2.rotate` uses one of three built-in codes — `cv2.ROTATE_90_CLOCKWISE`, `cv2.ROTATE_90_COUNTERCLOCKWISE`, or `cv2.ROTATE_180` — to perform an exact 90° or 180° rotation without any interpolation.

3.5 Flip (Feature 5)

`cv2.flip(img, 1)` mirrors the image left-to-right (horizontal flip). `cv2.flip(img, 0)` mirrors it top-to-bottom (vertical flip). The second argument is a flip code: 1 = horizontal, 0 = vertical.

3.6 Enhancement — Histogram Equalization and Gamma Correction (Feature 6)

Histogram Equalization: for grayscale images, `cv2.equalizeHist` directly redistributes intensities so the cumulative histogram is approximately flat across 0–255 — improving contrast in images that are mostly dark or mostly bright. For colour images the image is first converted to YCrCb colour space, equalization is applied only to the Y (luma) channel, and the result is converted back to BGR. This avoids unwanted colour shifts.

Gamma Correction: a look-up table (LUT) of 256 entries is built where entry $i = (i/255)^{(1/\gamma)} * 255$. When $\gamma > 1$ the curve is concave-up (output brighter than input); when $\gamma < 1$ the curve is concave-down (output darker). `cv2.LUT` applies the table to every pixel in $O(1)$ per channel using a precomputed mapping.

3.7 Gaussian Blur (Feature 7)

`cv2.GaussianBlur` convolves the image with a 2-D Gaussian kernel of size $k \times k$ (where k is odd, 3–21, selected by a slider). The kernel weights pixels by their distance from the centre using a bell-curve — pixels further away contribute less. Larger kernels produce stronger smoothing. Standard deviation is set to 0, so OpenCV auto-derives it from k .

3.8 Canny Edge Detection (Feature 8)

The Canny algorithm runs in four internal stages: (1) Gaussian smoothing to suppress noise, (2) Sobel gradient computation in x and y directions, (3) non-maximum suppression to thin edges to 1-pixel width, and (4) double-threshold hysteresis — pixels above the high threshold are definite edges, those below the low threshold are discarded, and those in between are kept only if connected to a definite edge. The low and high thresholds are exposed as sliders in the GUI with a constraint that $\text{low} < \text{high}$ is always maintained.

3.9 Compare — Live Histograms (Feature 9)

Two mini-histogram canvases (510 x 130 px) are always visible beneath the Original and Result image panels. They update automatically whenever a new result is generated. A separate Show Large Histogram button renders a 512 x 380 px detail view into the result panel for closer inspection without overwriting `self.current_img`.

3.10 Reset (Feature 10)

`self.current_img` is replaced with a fresh copy of `self.original_img` (via NumPy's array copy). `_display_result` is called, which re-displays the original and refreshes both histograms.

3.11 Save Image (Feature 11)

A `tk.filedialog.asksaveasfilename` dialog lets the user choose a file name and location. `cv2.imwrite` writes the current result NumPy array to disk in the format implied by the chosen extension (PNG or JPEG).

4. GUI Design Decisions

The application uses a dark theme (deep navy #1a1a2e background, accent red #e94560) to reduce eye strain and create visual contrast with the image panels. The control panel is scrollable to handle all buttons without requiring a very tall window. Hover effects (button background changes to red on mouse-over) provide visual feedback.

Sliders (ttk.Scale) are preferred over text-entry boxes for numeric parameters such as blur kernel size, gamma value, and Canny thresholds. This prevents invalid input and makes iterative adjustment fast. The Canny sliders enforce the low < high invariant in real time by automatically nudging the other slider when a conflict would arise.

The two-column image layout allows direct visual comparison between the original and the processed result. The live histograms make it immediately apparent how each operation changes the intensity distribution — for example, histogram equalization visibly flattens and spreads the histogram.

5. Setup and Installation

Python 3.8 or later is required. Install the dependencies with one command:

```
pip install opencv-python pillow matplotlib
```

Tkinter is bundled with the standard Python installer on Windows and macOS. On Linux it may need to be installed separately:

```
sudo apt-get install python3-tk
```

To launch the application:

```
python Final.py
```

6. Grading Criteria

The following grading breakdown was specified in the project requirements:

Criterion	Weight
All 11 features work correctly	50%
GUI is clean and easy to use	20%
Code is readable and Report printed	15%
Live demo and explanation	15%

All 11 required features are fully implemented and operational. The GUI follows a consistent design language with clear labels, hover feedback, and a scrollable control panel that accommodates all controls without clutter.

7. Conclusion

The Image Processing Desktop Application successfully implements all eleven required features in a polished, single-file Python program. The design separates concerns clearly: OpenCV handles all pixel-level computation, Tkinter manages the event loop and layout, Pillow bridges the two worlds, and Matplotlib provides publication-quality histogram visualisations rendered directly into the GUI without opening any external windows.

The live histogram strip beneath each image panel is an enhancement beyond the minimum requirements and provides immediate visual insight into how each operation transforms the intensity distribution. The application is ready for live demonstration and all team members can explain the features they implemented.